

Hackathon

Maxime Bourret et Alexandrine Dubé

14 Mai 2026

Lien repository Github
Projet déployé avec Vercel

1 Présentation du projet

1.1 Contexte et problème résolu:

1. Un organisateur crée un tournoi.
2. L'organisateur crée une ou plusieurs équipes dans ce tournoi.
3. Un joueur recherche une équipe disponible.
4. Le joueur envoie une demande d'adhésion.
5. Si le tournoi est payant, le joueur doit passer par Stripe.
6. L'organisateur voit les demandes reçues.
7. L'organisateur accepte ou refuse la demande.
8. Si la demande est acceptée, le joueur devient membre de l'équipe.

1.2 Contexte et problème résolu:

1. Organismes de tournoi.
2. Sportifs amateurs.
3. Personnes voulant voir des match de sport amateurs.

1.3 Fonctionnalité MVP livrées

- Authentification avec Clerk
- Synchronisation User Clerk vers Prisma (Via webhooks)
- Gestion des rôles : PLAYER, ORGANIZER, ADMIN

- Profil joueur : ville, sport, niveau, poste
- Dashboard organisateur
- Création et gestion des tournois
- Création et gestion des équipes
- Recherche de tournoi côté joueur
- Demandes d'adhésion
- Acceptation / refus des demandes
- Matches : création, liste, modification simple
- Paiement Stripe pour tournoi payant
- Section admin custom

1.4 Fonctionnalité Bonis livrées

- Déploiement Vercel
- Export CSV admin

2 Architecture technique

2.1 Stack utilisé:

- Clerk : Service d'authentification et gestion d'identité
- Stripe : Plateforme de paiement
- Zod : Validation de schémas TypeScript
- ESLint : Linting du code JavaScript/TypeScript
- tsx : Exécution TypeScript avec variables d'environnement
- Prisma : ORM moderne avec typage TypeScript complet
- Neon : Base de données PostgreSQL serverless
- adapter-neon : Adaptateur Neon pour Prisma
- TailwindCSS 4 : Framework CSS utilitaire pour le styling responsive
- shadcn/ui : Composants UI sans style intégrés
- Lucide React : Bibliothèque d'icônes open-source
- Next.js : Framework React full-stack avec App Router

- React : Bibliothèque de composants UI
- React DOM : Rendu DOM côté client
- TypeScript : Typage statique pour la sécurité du code

2.2 Schéma d'architecture:

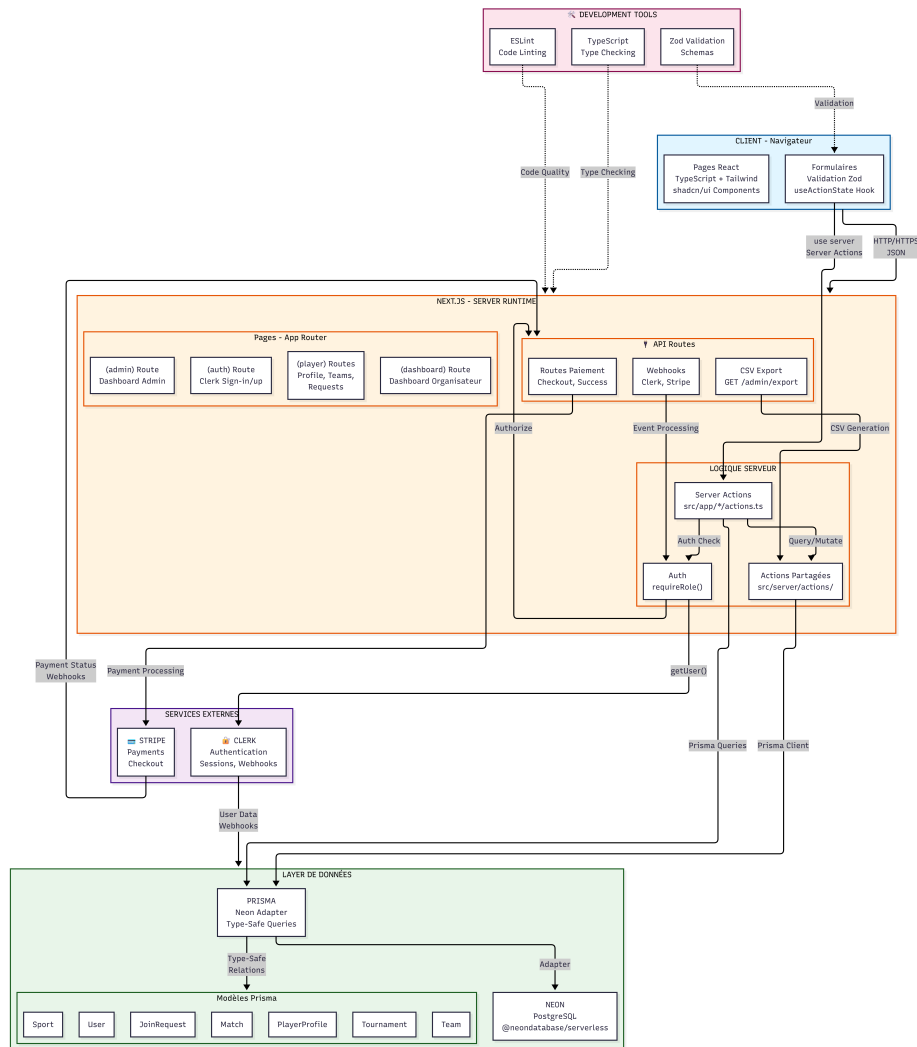


Figure 1: Diagramme d'architecture

2.3 Justifications des choix techniques:

2.4 Stack utilisé:

- Clerk : Gestion des comptes et des connexions avec partenaires sont gérés par Clerk. Intégration directe avec le code et permet facilement la synchronisation avec les webhooks paramétrables directement dans le Dashboard Clerk. Nous permet de ne pas implémenter de 0 l'authentification.
- Stripe : Permet de gérer les paiements sans avoir à nous-même l'implémenter dans notre code. Facilement intégrable dans le code, la validation des transactions est ensuite simple grâce à leurs webhooks.
- TypeScript et Zod : Typescript permet de facilement voir les erreurs de typages à la compilation et ainsi s'assurer que notre code est solide. En intégrant Zod par la suite, nous pouvons facilement créer des schemas de validations de données ce qui nous permet de ne pas manuellement refaire ces codes dans notre projet.
- Prisma : Création et initialisation de la BD simplifiée. Prisma génère un Client ce qui nous garantit que le code et notre schema de BD sont synchronisés.
- Neon : Version gratuite très accessible, permet de facilement faire le déploiement. Aucune configuration complexe à faire.
- TailwindCSS 4 : Styling CSS simplifié; intégration du dark mode se fait facilement. Avec un bon fichier CSS, nous n'avons pas besoin de manuellement utiliser des codes de couleurs dans notre code, ce qui améliore la lisibilité et la maintenabilité du code.
- shadcn/ui et Lucide React : Nous permet de facilement utiliser des composants UI solides, et donc de sauver du temps. Ils sont aussi personnalisables et peuvent donc facilement intégrer notre propre styling.
- Next.js : Utilisation de server actions sans créer de endpoint d'API nécessaire. Exécution côté serveur pour réduire la quantité de code à exécuter du côté client. Réduit la complexité du projet en n'ayant pas à séparer le frontend du backend.

3 Modèle de Données

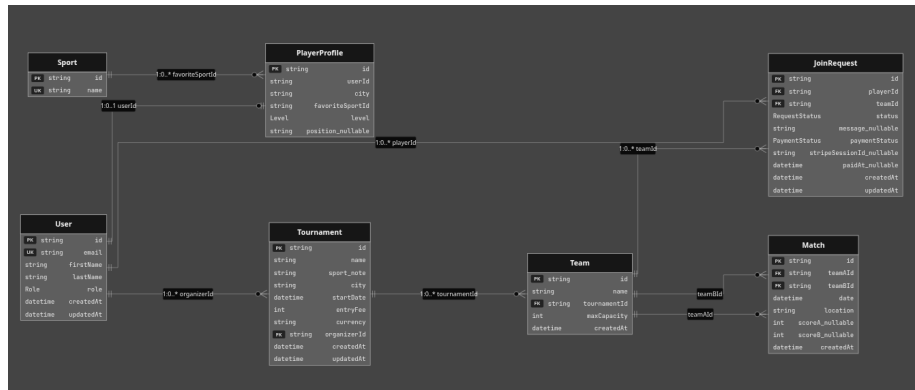
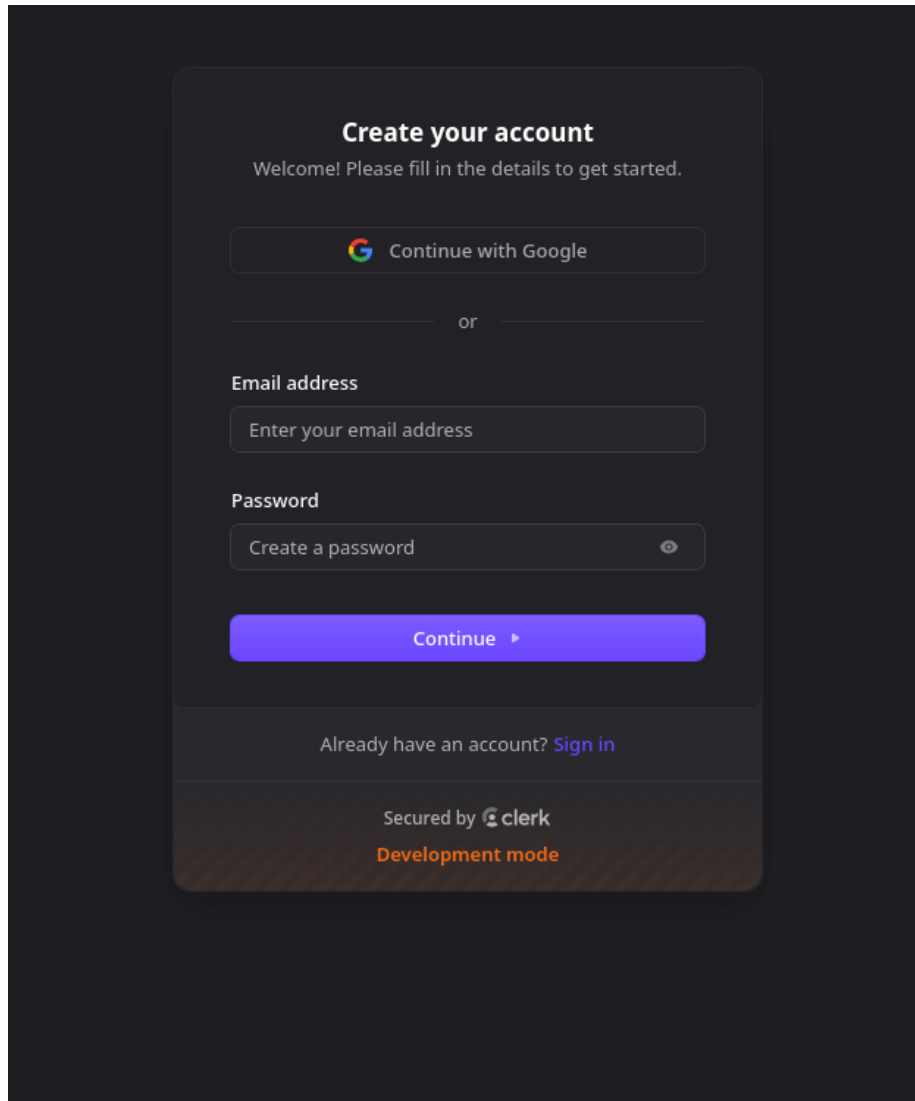


Figure 2: Diagramme Relationnel

4 Capture d'écran



The image shows a dark-themed user interface for creating an account. At the top, the heading "Create your account" is displayed in white, followed by the subtext "Welcome! Please fill in the details to get started." Below this, there is a button with the Google logo and the text "Continue with Google". A horizontal line with the word "or" in the center separates this from the next section. The "Email address" section has a label and a text input field with the placeholder "Enter your email address". The "Password" section has a label and a text input field with the placeholder "Create a password" and a small eye icon to toggle visibility. A large blue button with the text "Continue" and a right-pointing arrow is positioned below the password field. At the bottom of the form, there is a link that says "Already have an account? Sign in". The footer of the form indicates it is "Secured by clerk" and is in "Development mode".

Figure 3: Login

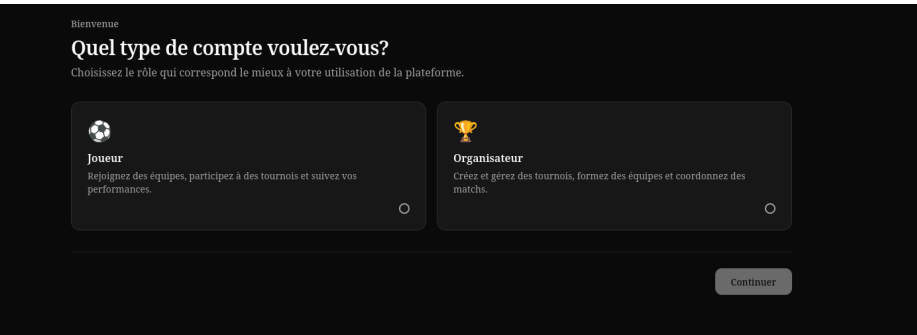


Figure 4: Role Selection

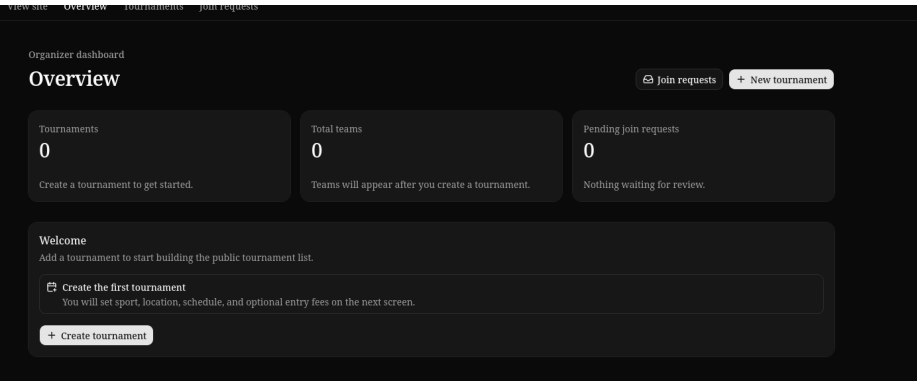


Figure 5: Dashboard

[Tournament platform](#) [Tournaments](#) [Matches](#) [My requests](#) [Dashboard](#)

[View site](#) [Overview](#) [Tournaments](#) [Join requests](#)

Organizer dashboard

Create tournament

Tournament details

You are creating this tournament as the organizer. It will be linked to your signed-in account.

Name

Tournoi

Sport

Basketball

City

Montreal

Start date

2026 - 05 - 15

Entry fee

10

Currency

CAD

Create tournament

Cancel

Figure 6: Formulaire tournoi

Tournament platformTournamentsMatchesMy requestsDashboard

View siteOverviewTournamentsJoin requests

Tournaments > Tournoi

SettingsTeamsMatches

Organizer dashboard

Create team

New team

Register a team under this tournament and set how many players it can hold.

Tournament: Tournoi · Basketball · Montreal

Team name

Northside Crushers

Max capacity

15

Maximum number of players who can join this team (1-200).

Create teamCancel

Figure 7: Formulaire Équipe

ournament platformTournamentsMatchesMy requestsDashboard

View siteOverviewTournamentsJoin requests

Tournaments > Tournoi

SettingsTeamsMatches

Organizer dashboard

Teams

Manage teams for Tournoi, Basketball in Montreal.

+ New team

Registered teams

Team	Capacity	Actions
Team 1	0 / 15 players	Edit
Team 2	0 / 15 players	Edit

Figure 8: Page équipe

The screenshot shows the 'Schedule match' form within the 'Tournament platform'. The navigation bar includes 'Tournament platform', 'Tournaments', 'Matches', 'My requests', and 'Dashboard'. The breadcrumb trail is 'Tournaments > Tournoi'. The form is titled 'Schedule match' and is part of the 'Organizer dashboard'. It contains the following fields:

- New match**: A section header with instructions: 'Pick two teams from this tournament, then set the time and location. Tournament: Tournoi - Basketball - Montreal'.
- Team A**: A dropdown menu with 'Team 1' selected.
- Team B**: A dropdown menu with 'Team 2' selected.
- Date and time**: A date picker showing '2026-05-25, --:-- --'.
- Location**: A text input field with 'Arena' entered.
- Buttons**: 'Create match' and 'Cancel' buttons at the bottom.

Figure 9: Formulaire Match

The screenshot shows the 'Find a tournament' page within the 'Tournament platform'. The navigation bar includes 'Tournament platform', 'Tournaments', 'Matches', 'My requests', and 'Dashboard'. The page is titled 'Find a tournament' and is part of the 'Public tournaments' section. It contains the following elements:

- Header**: 'Public tournaments' and 'Find a tournament'.
- Description**: 'Browse upcoming tournaments, review entry fees, and see current team and player counts.'
- Create tournament button**: A button labeled '+ Create tournament'.
- Tournament Card**: A card for 'Tournoi' (Basketball in Montreal) with a price tag of '10 CAD'. It displays:

Start date	Teams	Players
May 15, 2026	2	0

 It also mentions 'Organized by Maxime Bourret' and has a 'View details' button.

Figure 10: Recherche de tournoi

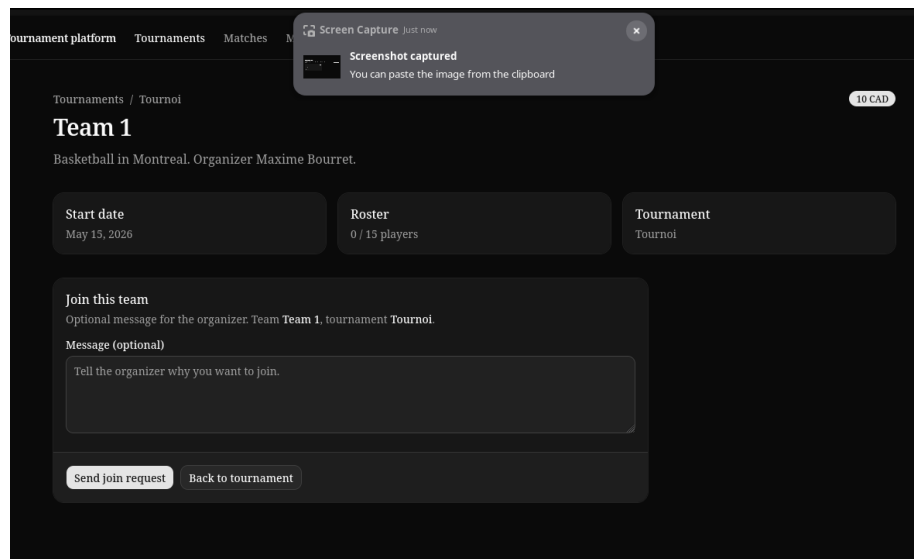


Figure 11: Demande d'admission

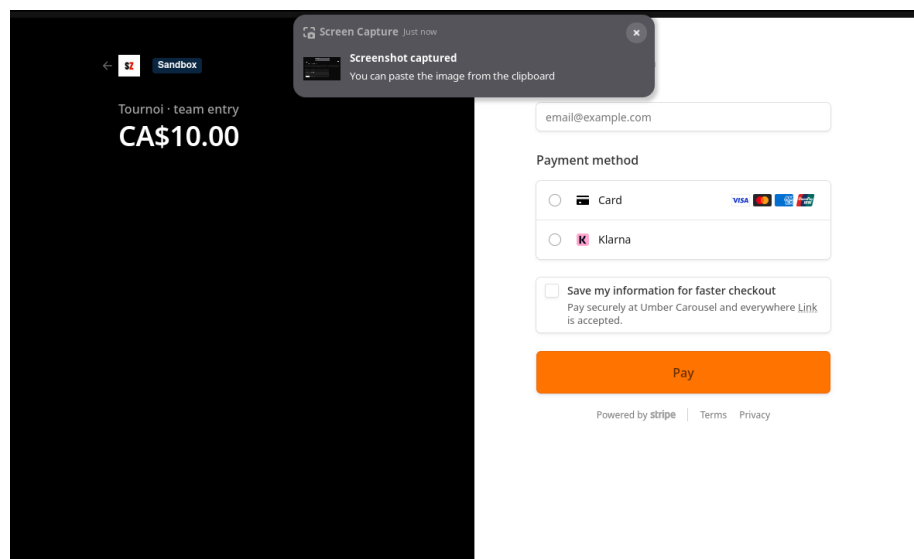


Figure 12: Checkout

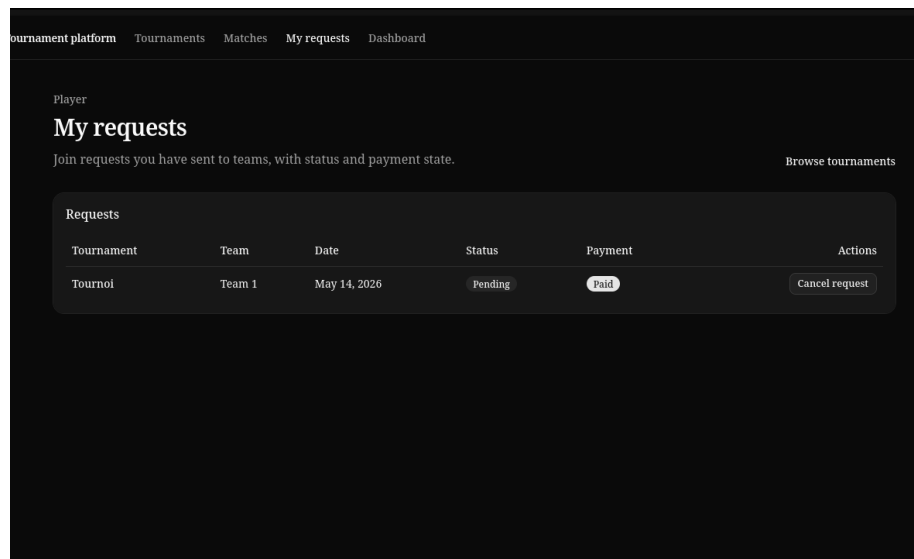


Figure 13: Requête coté joueur

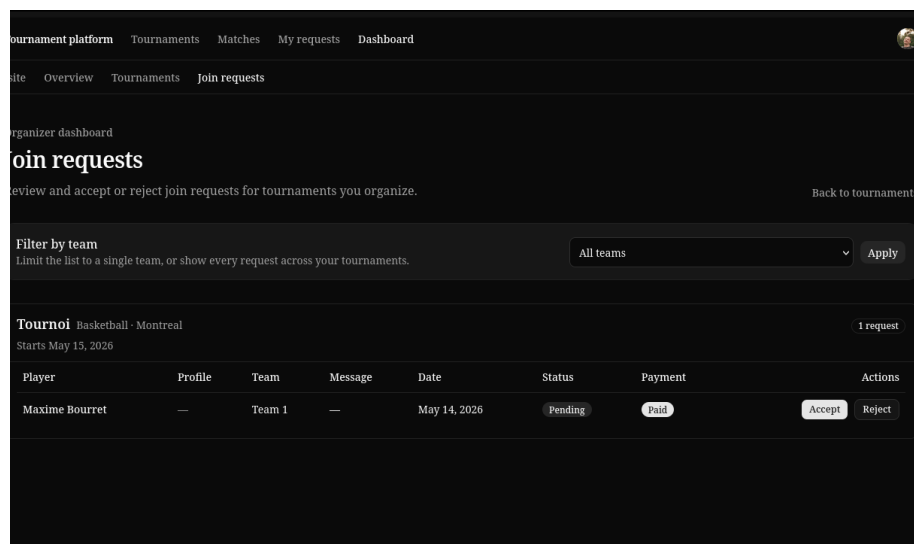


Figure 14: Requête coté organisateur

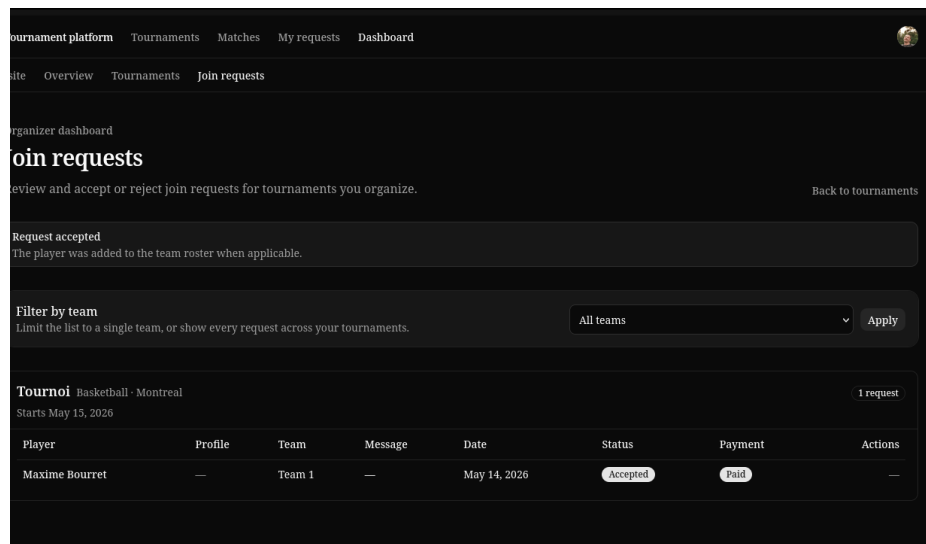


Figure 15: Requête accepté

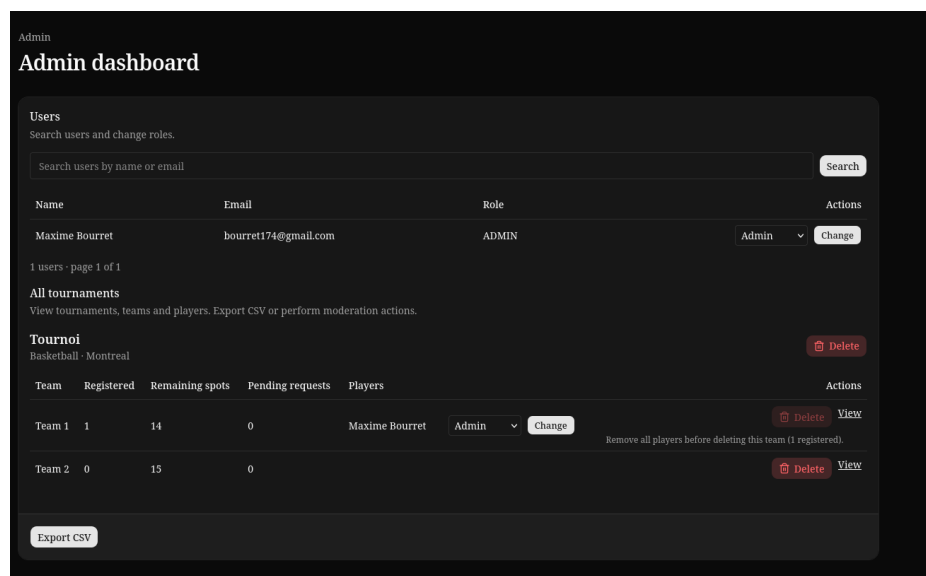


Figure 16: Dashboard Admin

5 Difficultés rencontrées et solutions

5.1 Principaux défis (techniques ou organisationnels) :

Avec nos expériences de travaux similaires et en équipe, nous savions que le défi principal d'un tel projet serait l'organisation, autant du temps que de l'équipe. Il était important pour nous de ne pas avoir à attendre après du code fait par l'autre, mais aussi de ne pas avoir à coder la même chose plusieurs fois si la façon de coder diffère.

5.2 Comment vous les avez résolus :

Dès le début du projet, nous avons créé un tableau GitHub Project pour créer des tickets couvrant toute la demande du PDF. À partir de ces tickets, nous avons pu discuter d'un plan de match logique. Maxime a donc commencé avec la gestion de Prisma, alors que j'ai commencé avec l'intégration de Clerk. Il était donc simple pour nous de tirer la branche main dans notre branche de travail au fur et à mesure que l'autre avançait. Nous avons aussi décidé de merge par Pull Requests que l'autre devait absolument review, ce qui nous permettait de toujours être à jour sur ce que l'autre faisait. En restant donc connectés dans un voice channel de Discord pendant toutes les périodes de travail, il était donc très simple de s'informer mutuellement du travail et des Pull Requests à venir, en plus de voir les tickets pris en charge par l'autre.

5.3 Ce que vous avez appris :

Avec une organisation solide et un bon plan de match, nous n'avons rencontré aucune embûche lors de ce projet. Nous avons réalisé qu'avoir confiance en la personne qui code sur le projet avec nous est primordial, et qu'une communication simple et bien organisée peut rendre un projet qui semble énorme maintenant tout simple et linéaire.